

```

"""
StegSim run.py - Main simulation loop.
Deterministic. Seed-locked. Receipt-producing.
Calls: init → propose → evaluate → apply → drift → metrics → report
"""

import json
import random
import hashlib
import numpy as np
from datetime import datetime, timezone
from pathlib import Path

from .config import SimConfig
from .model import (
    AgentState, EnvironmentState, SystemState, ViabilityMargin,
    state_hash,
)
from .agents import make_population, generate_proposals, apply_transition
from .governance import GovernanceEngine
from .drift import DriftEngine
from .receipts import ReceiptWriter, MetricsWriter
from .report import generate_report

def run_simulation(cfg: SimConfig) -> dict:
    """
    Run one complete simulation. Returns summary dict.
    All outputs written to cfg.output_dir / cfg.run_id /
    """
    # — Setup —————

    run_dir = Path(cfg.output_dir) / cfg.run_id
    run_dir.mkdir(parents=True, exist_ok=True)

    # Deterministic RNG - same seed = same run
    rng = random.Random(cfg.seed)
    np_rng = np.random.default_rng(cfg.seed)

    print(f"[StegSim] Run: {cfg.run_id} Scenario: {cfg.scenario} Seed: {cfg.seed}")
    print(f"[StegSim] Agents: {cfg.agents}, Steps: {cfg.steps}")

    # Write config
    with open(run_dir / "config.json", "w") as f:

```

```

    json.dump(cfg.to_dict(), f, indent=2)

# — Initialize state —————

agents = make_population(cfg.agents, cfg, rng, np_rng)

env = EnvironmentState(
    step=0,
    resource_scarcity=0.0,
    trust_decay_rate=cfg.drift.trust_decay_per_step,
    authority_drift_rate=cfg.drift.authority_staleness_per_step,
    policy_version=1,
    policy_lag=0.0,
    mutation_pressure=0.0,
    fragmentation=0.0,
    communication_fidelity=1.0,
    external_shock=0.0,
)

# Initial viability components
initial_components = {
    "trust_continuity":    cfg.initial.trust_mean,
    "authority_coherence": cfg.initial.authority_mean,
    "resource_balance":    cfg.initial.resource_balance_mean,
    "policy_freshness":    cfg.initial.policy_freshness,
    "mutation_pressure":    0.0,
    "fragmentation":        0.0,
}

viability = ViabilityMargin.compute(cfg.weights_dict(), initial_components)

state = SystemState(step=0, agents=agents, environment=env, viability=viability)

# — Initialize engines —————

governor = GovernanceEngine(cfg)
drifter   = DriftEngine(cfg, rng, np_rng)
receipts  = ReceiptWriter(str(run_dir / "receipts.jsonl"))
metrics   = MetricsWriter(str(run_dir / "metrics.jsonl"))

# — Tracking —————

initial_viability      = viability.composite
first_warning_step     = None
first_inadmissible_step = None
min_viability          = viability.composite
min_viability_step     = 0

```

```

total_proposals    = 0
total_allowed      = 0
total_denied       = 0
total_flagged      = 0
total_fail_closed  = 0
total_lg_disagree  = 0

peak_lg_disagree_rate = 0.0

print(f"[StegSim] Initial viability: {viability.composite:.4f} ({viability.st

# — Main loop —————

for step in range(cfg.steps):
    state.step = step

    # 1. Generate proposals
    proposals = generate_proposals(state, rng)

    # 2. Evaluate each proposal
    step_allowed = step_denied = step_flagged = step_fail = step_lg = 0

    for proposal in proposals:
        result = governor.evaluate(proposal, state)
        receipts.write(result, cfg.run_id)
        metrics.accumulate(result)

        total_proposals += 1
        if result.decision == "ALLOW":
            step_allowed += 1
            total_allowed += 1
            apply_transition(state, proposal)
        elif result.decision == "DENY":
            step_denied += 1
            total_denied += 1
        elif result.decision == "FLAG":
            step_flagged += 1
            total_flagged += 1
            # Flagged transitions are applied but recorded
            apply_transition(state, proposal)
        else: # FAIL_CLOSED
            step_fail += 1
            total_fail_closed += 1

        if result.local_global_disagree:
            step_lg += 1
            total_lg_disagree += 1

```

```

# 3. Apply drift
state = drifter.apply(state)

# 4. Emit metrics every N steps
if step % cfg.report_every == 0 or step == cfg.steps - 1:
    m = metrics.flush_step(step, state, cfg.run_id)
    lg_rate = m["local_global_disagreement_rate"]
    if lg_rate > peak_lg_disagree_rate:
        peak_lg_disagree_rate = lg_rate

    v = state.viability.composite
    if v < min_viability:
        min_viability = v
        min_viability_step = step

    if first_warning_step is None and v < cfg.thresholds.warning:
        first_warning_step = step
        print(f"[StegSim] Step {step:4d}: WARNING threshold crossed - via

    if first_inadmissible_step is None and v < cfg.thresholds.inadmissibl
        first_inadmissible_step = step
        print(f"[StegSim] Step {step:4d}: INADMISSIBLE threshold crossed

    if step % (cfg.report_every * 5) == 0:
        print(f"[StegSim] Step {step:4d}: V={v:.4f} ({state.viability.sta
            f"LG={lg_rate:.3f} A={step_allowed} D={step_denied}")

final_viability = state.viability.composite
print(f"[StegSim] Final viability: {final_viability:.4f} ({state.viability.st

# — Summary —————

summary = {
    "schema":          "vbds.summary.v1",
    "run_id":          cfg.run_id,
    "scenario":         cfg.scenario,
    "seed":            cfg.seed,
    "agents":          cfg.agents,
    "steps":           cfg.steps,
    "generated_at":    datetime.now(timezone.utc).isoformat(),

    # Viability
    "initial_viability": round(initial_viability, 4),
    "final_viability":   round(final_viability, 4),
    "min_viability":     round(min_viability, 4),
    "min_viability_step": min_viability_step,

```

```

        "first_warning_step":      first_warning_step,
        "first_inadmissible_step": first_inadmissible_step,
        "final_viability_status":  state.viability.status,

        # Decisions
        "total_proposals":         total_proposals,
        "accepted_count":          total_allowed,
        "denied_count":            total_denied,
        "flagged_count":           total_flagged,
        "fail_closed_count":       total_fail_closed,

        # Key metrics
        "total_local_global_disagree": total_lg_disagree,
        "peak_local_global_disagreement_rate": round(peak_lg_disagree_rate, 4),
        "total_receipts":           receipts.count(),

        # Determinism verification
        "final_state_hash":        state.full_hash(),
        "config_hash":             state_hash(cfg.to_dict()),
    }

    with open(run_dir / "summary.json", "w") as f:
        json.dump(summary, f, indent=2)

    # — Report —————

    generate_report(str(run_dir), cfg.to_dict(), summary)
    print(f"[StegSim] Report written to {run_dir}/REPORT.md")
    print(f"[StegSim] Receipts: {receipts.count():,}")

    return summary

# — Determinism verification —————

def verify_determinism(cfg: SimConfig, n_runs: int = 2) -> bool:
    """
    Run the simulation n_runs times and confirm same final_state_hash.
    Returns True if deterministic.
    """
    print(f"[StegSim] Verifying determinism ({n_runs} runs)...")
    hashes = []
    for i in range(n_runs):
        from dataclasses import replace
        cfg_i = SimConfig.from_dict(cfg.to_dict())
        cfg_i.run_id = f"{cfg.run_id}-verify-{i}"
        summary = run_simulation(cfg_i)

```

```
hashes.append(summary["final_state_hash"])

ok = len(set(hashes)) == 1
print(f"[StegSim] Determinism: {'PASS' if ok else 'FAIL'} - hashes: {set(hashes)}")
return ok
```